

CS 150-01 Theoretical CompSci Toolkit

V. Podolskii

TR 12:00-1:15, Room To Be Announced

[E+](#) Block

In this course, we will explore various methods and techniques that are useful in various areas of computation theory. We will see how combinatorial and probabilistic technique, polynomial approach, Fourier analysis and linear algebraic technique can be applied to solve various problems in theoretical computer science. We will illustrate the techniques by the examples in theory of algorithms, computational complexity, learning theory, property testing and other areas.

Prerequisite: Prior completion of CS 61 or graduate standing.

Recommendations: CS 170 or equivalent is highly recommended, but is not required. CS 160 or equivalent would be helpful, but is not necessary. MATH 70 (Linear algebra) is recommended to have prior to this course, but this is not a prerequisite.

CS 150-02 Advanced Programming Languages

G. Wei

TR 4:30-5:45p, Room To Be Announced

[L+](#) Block

This graduate course explores advanced topics in programming languages. The primary goal of this course is to prepare students to explore the literature and perform research in this field. By the end of the course, students will be able to understand and implement results from research papers, as well as develop their own systems.

Topics include, but are not limited to: operational semantics (small-step reduction semantics, abstract machines, natural semantics), type systems (polymorphism, subtyping, type inference, refinement types, ownership), effects (continuations, algebraic effects and handlers, type-and-effect systems), analysis & transformation (CPS/ANF transformation, multi-stage programming, partial evaluation, symbolic execution), etc. The range of topics remains flexible and will be driven by student interests.

The course format includes a combination of lectures, student-led paper presentations, and project work. Each student is expected to actively participate in discussions, present research papers, lead discussions, and conduct a research project with a written report. Undergraduates who have completed CS105 are welcome to enroll.

Prerequisite: CS 105 or equivalent.

CS 150-03 Quantum Computer Science

S. Mehraban

TR 1:30-2:45, Room To Be Announced

[H+](#) Block

This is an elementary-level introduction to quantum computing through the computer science lens. One of the goals of this course is to present a language of quantum mechanics that is accessible to students working in computer science and vice versa. Topics include Hilbert spaces, the Dirac notation, the Schrödinger equation, quantum circuits, quantum Fourier transforms, quantum algorithms, and physical realizations of quantum computers. Students from different areas of engineering and sciences such as computer science, physics, electrical engineering, mathematics, or chemistry who wish to learn about the computer science foundations of quantum computers can benefit from this class.

Prerequisite: Linear algebra MATH 70 (Linear Algebra) or equivalent. Basic familiarity with discrete math, algorithms, and calculus is also recommended.

CS 150-04 Generative Models

L. Liu

MW 10:30-11:45, Room To Be Announced

[E+](#) Block

Generative models have achieved remarkable success across various applications, including Large Language Models (LLMs) and multimodal generative AI. This course introduces the probabilistic foundations of deep generative models and provides hands-on experience in coding these models. We begin with simple distributions, exploring different distribution forms and their sampling procedures. Then, we transition to generative models based on neural networks. While these models are highly flexible and capable of capturing complex distributions, they require specialized learning algorithms for effective training. This course covers a range of these algorithms, with a primary focus on sequential models, diffusion-based models, and flow matching—key techniques underpinning modern generative AI. By the end of the course, students will develop a solid theoretical understanding of generative models and gain practical experience in implementing them.

Prerequisite: Prerequisites: 1) programming: students should be comfortable with python programs. Students will write plenty of programs with numpy and pytorch in this course. 2) probability theory: students should have knowledge of probability theory, which includes concepts of probability (density) functions, conditional/marginal probabilities, log-likelihood, and simple distributions (uniform, Gaussian distribution). 3) linear algebra and calculus: students should know basic operations over matrices and vectors (eigenvalue decomposition is not needed). Students should know the concepts of gradients and partial derivatives.

CS 150-05 AI Foundations

M. Scheutz

MW 3:00-4:15, Room To Be Announced

[I+](#) Block

This course will introduce basic mathematical and computational concepts required for studying more advanced artificial intelligence algorithms and architectures.

CS 150-07 Algorithmic Music Composition

[R. Townsend](#)

MW 3:00-4:15, Room To Be Announced

[I+](#) Block

This course will explore music composition methods that use a purely algorithmic approach: examples include running a monte carlo method for a melody, deriving a chord progression from a grammar, or generating a whole piece with a cellular automaton. Students will compose their own pieces of music using these algorithmic techniques, and implement their compositions as programs. This course does not cover software tools for music generation (e.g., garageband, audacity, SuperCollider) nor how computers generate sound fundamentally.

Prerequisite: Prerequisite: Comfortable reading sheet music in the treble clef and basic music theory knowledge (pitches, rhythm, intervals, keys, scale degrees)

CS 150-08 Scientific Machine Learning

A. Patra

Scientific Machine Learning: (in reality the use of Machine Learning and AI for science investigations) SciML is an integration of traditional scientific computing and machine learning, which combines mechanistic models with data-driven reasoning, presented as a unified set of abstractions and a high-performance implementation. Science usage places different demands on ML tools in terms data availability, constraints and questions of interest and requires new approaches which are often generalizing to other traditional applications. For

instance, while traditional deep learning methodologies have had difficulties with scientific issues like stiffness, interpretability, and enforcing physical constraints, this blend with data driven analytics and numerical analysis and differential equations has evolved into a field of research with new methods, architectures, and algorithms which overcome these problems while adding the data-driven automatic learning features of modern deep learning. Many successes have already been found, with tools like [physics-informed neural networks](#), [universal differential equations](#), [solvers for high dimensional PDEs](#), and [neural surrogates](#) showcasing how ML/AI can greatly improve scientific modeling practice and support decision making where science must be used e.g. climate adaptation.

Course will also build in training on software performance engineering.